

好的，根据您提供的项目文件、需求文档以及参考图片，我为您撰写了对应的项目文档。

4. 系统概要设计

4.1 系统整体架构设计

本“校园健康管理卫士”App采用单体客户端架构，所有业务逻辑、数据处理和存储均在Android客户端本地完成。其核心架构可分为三个主要层次：

- **表现层 (UI Layer):** 负责与用户进行交互，展示数据并接收用户输入。此层由一系列的 `Activity` 和 `Fragment` 组成，例如 `LoginActivity`（登录）、`MainActivity`（主功能界面）、`HomeFragment`（首页）等。XML布局文件定义了每个页面的静态结构，而Java类则负责处理动态交互逻辑。
- **业务逻辑层 (Business Logic Layer):** 作为表现层和数据访问层之间的桥梁，负责处理核心业务规则。例如，在用户登录时，`LoginActivity` 会调用业务逻辑来验证用户名和密码的有效性；在提交请假申请时，`LeaveApplicationActivity` 会封装用户输入的数据并准备存入数据库。
- **数据访问层 (Data Access Layer):** 专门负责与本地SQLite数据库进行交互。`DatabaseHelper.java` 是该层的核心，它封装了所有SQL操作，包括数据库的创建、升级以及对各个数据表（如用户表、请假表等）的增、删、改、查（CRUD）方法。业务逻辑层通过调用 `DatabaseHelper` 的方法来持久化和检索数据，实现了与具体SQL语句的解耦。

4.2 数据库概念设计

在需求分析的基础上，系统中的核心实体及其关系（E-R图）设计如下：

- **实体 (Entities):**
 - **用户 (User):** 包含ID、用户名、密码和角色（学生、老师、医生、管理员）。
 - **每日打卡 (CheckIn):** 记录用户的打卡行为，包含ID、用户ID和打卡日期。
 - **饮食记录 (Diet):** 记录用户的饮食，包含ID、用户ID、餐别、食物内容和日期。
 - **运动记录 (Exercise):** 记录用户的运动，包含ID、用户ID、运动类型和日期。
 - **就诊预约 (Appointment):** 记录用户的预约信息，包含ID、用户ID、科室、时间和病情描述。
 - **请假申请 (LeaveRequest):** 记录学生的请假，包含ID、用户ID、原因、时间和状态。
 - **消息通知 (Notification):** 记录由老师发布的通知，包含ID、标题和内容。
- **关系 (Relationships):**
 - 一个 **用户** 可以有多条 **每日打卡**、**饮食记录**、**运动记录**、**就诊预约** 和 **请假申请** 记录。这些都是一对多的关系。
 - **消息通知** 实体与用户没有直接的数据库关联，它是一个面向所有用户的公告板。

4.3 数据库逻辑设计

根据概念设计，我们将实体和关系转化为具体的数据库表结构。

1. 用户信息表 (user_table)

- `id`: INTEGER, PRIMARY KEY
- `username`: TEXT, UNIQUE, NOT NULL
- `password`: TEXT, NOT NULL
- `role`: TEXT, NOT NULL

2. 每日打卡表 (checkin_table)

- `user_id`: INTEGER
- `checkin_date`: TEXT

3. 饮食记录表 (diet_table)

- `id`: INTEGER, PRIMARY KEY
- `user_id`: INTEGER
- `meal_type`: TEXT
- `food_content`: TEXT
- `record_date`: TEXT

4. 运动打卡表 (exercise_table)

- `id`: INTEGER, PRIMARY KEY
- `user_id`: INTEGER
- `exercise_type`: TEXT
- `exercise_date`: TEXT

5. 就诊预约表 (appointment_table)

- `id`: INTEGER, PRIMARY KEY
- `user_id`: INTEGER
- `department`: TEXT
- `appointment_time`: TEXT
- `description`: TEXT

6. 请假申请表 (leave_table)

- `id`: INTEGER, PRIMARY KEY
- `user_id`: INTEGER
- `reason`: TEXT
- `leave_time`: TEXT
- `status`: TEXT

7. 消息通知表 (notification_table)

- `id`: INTEGER, PRIMARY KEY
- `title`: TEXT
- `content`: TEXT

5. 系统详细设计

5.1 用户与多角色管理功能设计

该功能是整个系统的基础，实现了用户的身份认证和权限区分。

- **注册流程:** 用户在 `RegisterActivity` 中输入用户名、密码，并通过 `RadioGroup` 选择角色（学生、老师、医生、管理员）。点击注册后，系统会调用 `DatabaseHelper` 的 `addUser` 方法，将用户信息存入 `user_table`。在存入前，会先查询数据库确保用户名未被占用。

- **登录流程:** 用户在 `LoginActivity` 中输入账号密码。系统调用 `dbHelper.getUserByUsername()` 方法查询 `user_table`。验证成功后, 会将用户的ID、用户名和角色存入 `SharedPreferences`。
- **多角色路由:** 登录成功后, 系统会根据从 `SharedPreferences` 中读取的用户角色, 通过 `Intent` 跳转到不同的主界面:
 - **学生/老师:** 跳转到 `MainActivity`。
 - **医生:** 跳转到 `DoctorMainActivity`。
 - **管理员:** 跳转到 `AdminMainActivity`。
- **退出登录:** 在 `ProfileFragment` 中, 点击“退出登录”会清除 `SharedPreferences` 中的用户数据, 并跳转回 `LoginActivity`。

5.2 请假及审批功能设计

该功能实现了学生和老师之间的请假审批闭环。

- **学生请假流程:**
 1. 学生在 `HomeFragment` 点击“请假申请”按钮, 跳转到 `LeaveApplicationActivity`。
 2. 在 `LeaveApplicationActivity` 中填写请假时间和原因, 点击“提交申请”。
 3. 系统创建一个 `LeaveRequest` 对象, 设置其初始状态为“待审批” (`LeaveRequest.STATUS_PENDING`)。
 4. 调用 `dbHelper.addLeaveRequest()` 方法, 将申请记录存入 `leave_table`。
- **老师审批流程:**
 1. 老师在 `HomeFragment` 点击“审批请假”按钮, 跳转到 `LeaveApprovalActivity`。
 2. `LeaveApprovalActivity` 加载时, 调用 `dbHelper.getAllLeaveRequests()` 获取所有请假记录。
 3. 通过自定义的 `LeaveRequestAdapter`, 将每条申请显示在 `ListView` 中。适配器会根据申请人的 `user_id` 查询 `user_table` 以显示学生姓名。
 4. 对于状态为“待审批”的申请, 列表项会显示“批准”和“拒绝”按钮。点击按钮会更新该条 `LeaveRequest` 对象的状态为“已批准”或“已拒绝”, 并调用 `dbHelper.updateLeaveRequestStatus()` 更新数据库。
 5. 更新成功后, 页面会刷新列表, 按钮消失, 状态文本更新。

5.3 用户账户管理（管理员）功能设计

该功能为管理员提供了管理所有系统用户的能力。

- **查询所有用户:** `UserManageActivity` 启动时, 会调用 `dbHelper.getAllUsers()` 方法, 获取 `user_table` 中的所有用户数据, 并展示在一个 `ListView` 中。
 - **添加新用户:** 点击“添加新用户”按钮, 会弹出一个包含输入框（用户名、密码）和 `Spinner`（角色选择）的 `AlertDialog`。填写完毕后, 系统调用 `dbHelper.addUser()` 方法创建新用户。
 - **编辑用户:** 点击列表中的任意用户, 在弹出的选项选择“编辑”, 会复用上述 `AlertDialog`, 但会预先填好该用户的当前信息。确认修改后, 系统调用 `dbHelper.updateUser()` 方法更新数据库。
 - **删除用户:** 点击用户并选择“删除”, 在二次确认后, 系统调用 `dbHelper.deleteUser()` 方法, 根据用户ID从数据库中删除该条记录。每次操作成功后, 用户列表都会自动刷新。
-

6. 系统实现

6.1 多角色登录及路由实现

多角色登录的核心在于登录成功后，根据从数据库获取的角色信息，动态地决定跳转到哪个 Activity。这一逻辑在 LoginActivity.java 的 loginUser 方法中实现。

```
// 在 loginUser() 方法中
User user = dbHelper.getUserByUsername(username);
```

6.2 数据库增删改查操作实现

所有数据库操作都封装在 `DatabaseHelper.java` 类中，该类继承自 `SQLiteOpenHelper`。通过 `getWritableDatabase()` 或 `getReadableDatabase()` 获取数据库实例，然后使用 `ContentValues` 对象和 `insert()`、`update()`、`delete()`、`query()` 等方法来执行 SQL 操作。

以“添加请假申请”为例，`addLeaveRequest` 方法的实现如下：

```
```java
// DatabaseHelper.java
public long addLeaveRequest(LeaveRequest leaveRequest) {
 SQLiteDatabase db = this.getWritableDatabase();
 ContentValues values = new ContentValues();
 values.put(KEY_USER_ID, leaveRequest.getUserId());
 values.put(KEY_REASON, leaveRequest.getReason());
 values.put(KEY_LEAVE_TIME, leaveRequest.getLeaveTime());
 values.put(KEY_STATUS, leaveRequest.getStatus());
 long id = db.insert(TABLE_LEAVE, null, values);
 db.close();
 return id;
}
```

### 6.3 动态UI实现

系统通过检查登录用户的角色，动态地显示或隐藏特定的UI元素，从而实现权限控制。例如，在 HomeFragment.java 中，根据 userRole 变量决定是否显示“请假申请”（学生）或“审批请假”（老师）按钮。

```
// HomeFragment.java
private void checkUserRole() {
 if ("老师".equals(userRole)) {
 btnLeaveApproval.setVisibility(View.VISIBLE);
 // ... 设置点击事件 ...
 } else if ("学生".equals(userRole)) {
 btnLeaveApplication.setVisibility(View.VISIBLE);
 // ... 设置点击事件 ...
 }
}
```

## 6.4 ListView 与自定义适配器实现

为了在列表中展示复杂的布局（如包含多个文本和按钮的请假条），系统使用了自定义 `ArrayAdapter`。以 `LeaveRequestAdapter` 为例，它重写了 `getView()` 方法。在该方法中：

1. 加载自定义的列表项布局 `list_item_leave_request.xml`。
2. 获取当前位置的 `LeaveRequest` 对象。
3. 通过 `dbHelper.getUserById()` 查询并显示学生姓名。
4. 将请假时间、原因和状态填充到对应的 `TextView` 中。
5. 根据请假状态（`STATUS_PENDING`）决定是否显示“批准”和“拒绝”按钮。
6. 为按钮设置点击监听器，在监听器中执行数据库更新操作，并调用 `LeaveApprovalActivity` 的公共方法刷新整个列表，实现数据的实时同步。